

0.20.3. La fonction `deplace_y()` — avancer sur une colonne

`#include <deplace_y>` : `deplace_y(x, y, tuile, pas)` fait descendre la tuile de « pas » lignes (ou monter si pas est négatif).

1. Ce qu'il faut comprendre

Ce qu'elle fait : comme `deplace_x()`, mais **de haut en bas** : `pas` positif descend, négatif monte. Elle s'arrête au bord de l'écran.

La ligne à écrire : `#include <deplace_y>`.

Ses arguments : colonne, ligne, tuile, pas ; seule sur sa ligne, elle range l'arrivée dans la variable de la ligne.

Ce qu'elle coûte : environ **220 octets**, plus la police entière.

Essaie : `deplace_y(10, y, ALPHABET[0], -6);` : le A remonte.

On retrouve `deplace_y` dans 12 leçons

[0.21](#), [0.21.1](#), [0.21.2](#), [0.22](#), [0.22.1](#), [0.22.2](#), [0.24](#), [0.24.1](#), [0.24.2](#), [0.25](#), [0.25.1](#), [0.25.2](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1068 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	60 o	← la console, à l'allumage – la mise en route de la console
deplace_y()	158 o	← main() – une fonction fournie par le langage
main()	28 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases – attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() – efface des cases
CopierTuiles	18 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
Tuiles	704 o	← CopierTuiles – la police entière

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 2	26 o sur place, dans deplace_y() + AdresseCase (10 o)
effacer()	× 1	15 o sur place, dans deplace_y() + AdresseCase (10 o), EffacerCases (12 o)
image()	× 2	6 o sur place, dans deplace_y(), main() + AttendreImage (28 o)

```
Tes fonctions appelées :  
deplace_y() x 1 19 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <deplace_y> : avancer sur une colonne ----  
// Le A descend de la ligne 2 à la ligne 10.  
  
#include <deplace_y> // deplace_y() : avancer sur la colonne  
#include <ALPHABET>
```

2. UNE VARIABLE — Y

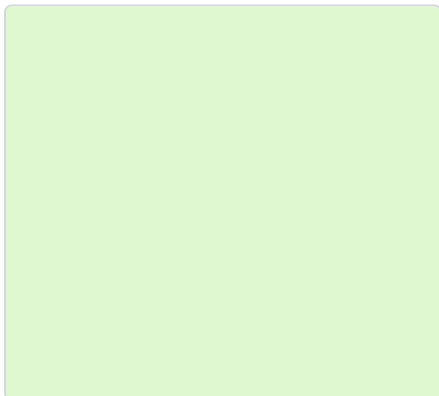
```
uint8_t y = 2;
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

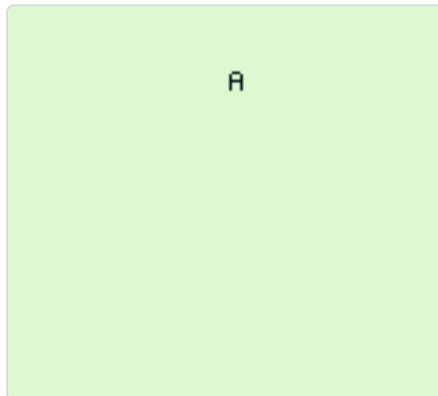
```
int main() {  
    deplace_y(10, y, ALPHABET[0], 8); // 8 pas vers le bas ; y reçoit 10  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

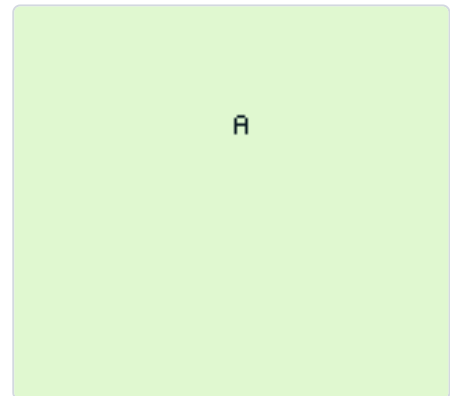
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un A qui descend de huit lignes.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A finit en (10, 10) (10,10)

✓ y a reçu l'arrivée : 10

Ensuite : **0.21. Descendre avec `deplace_y`** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++